



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

DE52 Software Engineering

11 mars 2025

**Software life cycle, requirements and
general design**

Table of contents

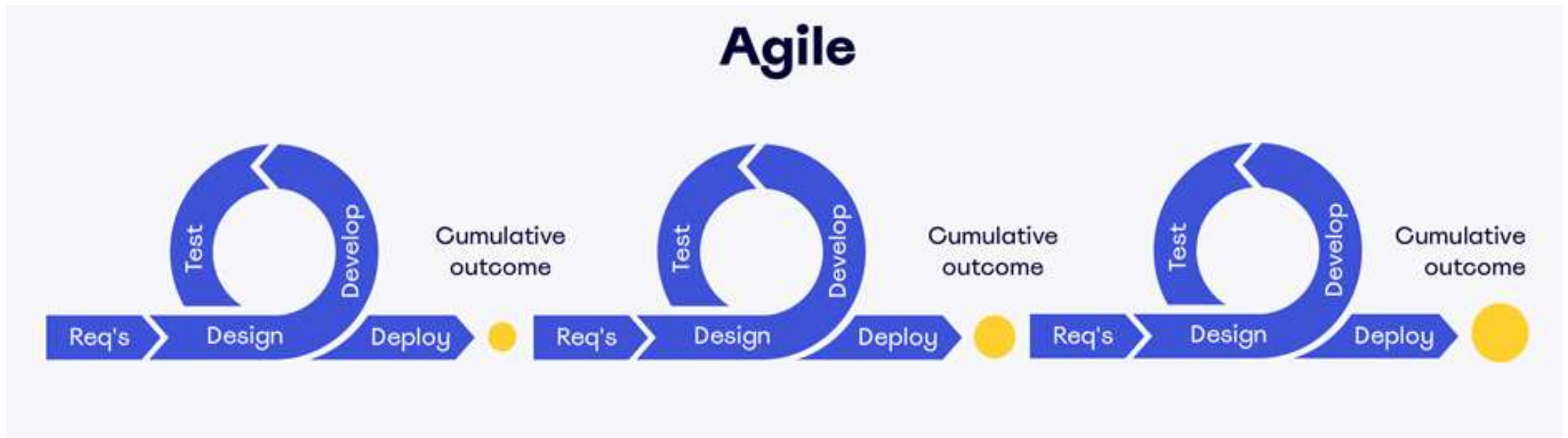
- Agile and Scrum
- Software requirements and general design
 - User Stories
 - Conceptual Mockups
 - Class, Responsibility and Collaborator (CRC) cards
 - Trade offs and compromises



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Agile and Scrum

Agile



Agile approaches for software development

- Definition: an agile method is an **iterative** and **incremental** methodology for software development carried out in a **collaborative** spirit with just enough **formalism**. It emphasizes flexibility, customer collaboration, and rapid delivery of functional software.
- Some approaches : Scrum, Kanban, eXtreme programming
- Agile approaches: common principles *[Agile Manifesto (2001)]*
 - Importance of people and their interactions more than processes and tools
 - Software that works rather than exhaustive documentation
 - Collaboration with the customer rather than contract negotiation
 - Responding to change rather than following a predefined plan

Agile and Scrum

- Agile project development entails a set of “methods and practices based on the values and principles expressed in the Agile Manifesto,”. This comprehensive set may refer to self-discipline, cross-functional teams, and teamwork
- The term Scrum refers to the framework or approach that allows for Agile development methodology to be implemented
- Analogy: the relationship between a diet/lifestyle (vegetarian or vegan) and a recipe you use to implement the diet
- 66% of the frameworks used to implement the Agile workflow are Scrum

Agile and Scrum: Framework details

- Scrum has three different roles, all of which are mandatory, for Scrum to be successful:
 - **Product owner:** is the voice of the user and has the final word as far as what the project needs,
 - **Scrum master:** guides the team through the Scrum values
 - **Development team:** developers, designers, testers, and operations engineers

- The Product Backlog:
 - Creating the *product backlog* is an essential step as it involves ordering by importance what the project needs.
 - The backlog is also the part that is always evolving, adapting to new needs. It's never complete.
 - Creating the product backlog is the **product owner's** responsibility

Agile and Scrum: Framework details

■ Sprint Planning

- A *sprint* is a time-limited cycle during which the team needs to complete a certain number of tasks from the *backlog*. A sprint is from 2 to 4 weeks.
- When planning a sprint, you need to select specific tasks for that particular sprint and assign them to the team members.

■ The Work Process

- Team members have been assigned clear tasks, and the progress of completing that task is reviewed daily. The *Daily Scrum Meeting* should last less than a quarter of an hour (15 min) and focus on providing answers to these three questions:
 - What part of the task did a team member work on the day before?
 - What will that team member be working on today?
 - What can be done to assist team members with anything blocking their work today?

Agile and Scrum: Framework details

■ Sprint Reviews

- Work is reviewed with the client at the end of each Scrum sprint.
- It is the decisive moment when the team communicate the tasks accomplished and review the work done to complete those tasks.

■ Process Review or Sprint retrospective

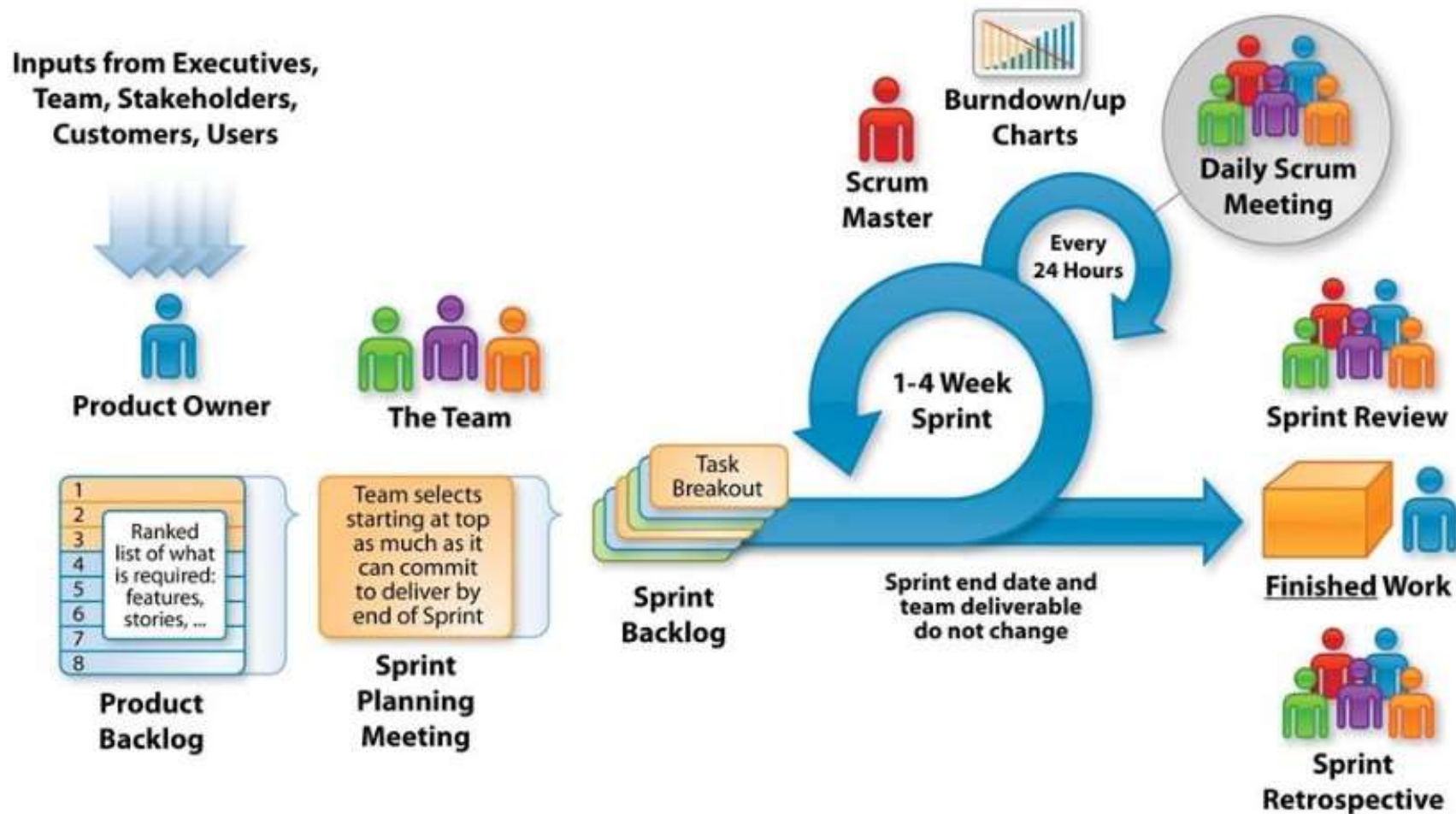
- There is also a *retrospective meeting*, without the client, that Scrum entails when the team takes a good look at the work process and identifies solutions to streamline the work process.

■ Repeat the Process

- Although this is the last step, it is the beginning of a new work cycle.
- Once you have completed your first Scrum Sprint, you need to move on to the next sprint by assigning new tasks from the product backlog to be completed by team members.

Agile and Scrum

The Agile - Scrum Framework



A **Sprint Backlog** includes tasks from the Product Backlog assigned to this Sprint

A **burndown chart** is a graphical representation of work left to do versus time

A **Scrum Master** is the key facilitator responsible for ensuring that the Scrum process is understood, implemented correctly, and continuously improved



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Stages of the life cycle

The stages of the life cycle

1. Preliminary study
2. Requirements phase (Specifications)
3. General (or conceptual) Design
4. Detailed design
5. Development
6. Unit Testing
7. Integration and verification
8. Validation
9. Operation & Maintenance



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Software requirements and general design

Software Requirements and General Design

- User Stories
- Conceptual Mockups
- Class, Responsibility and Collaborator (CRC) cards
- Trade offs and compromises



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

User Stories

A requirement as a user story

- A user story is simply a requirement, often from the perspective of an end-user, which is stated in natural language

As a _____, I want to _____ so that _____

- Parts:
 - First blank: User role
 - Second blank: Goal that the user role wants to achieve
 - Third blank: Reason why the user role wants this goal

User story: An example

- Nouns correspond to objects in your software and verbs can help you identify the requirements that your objects might have

As an **online shopper**, I want to **add** an **item** to my **shopping cart**,
so that I can **purchase** it

- A user story can also help you discover connections between objects
- Object-oriented thinking

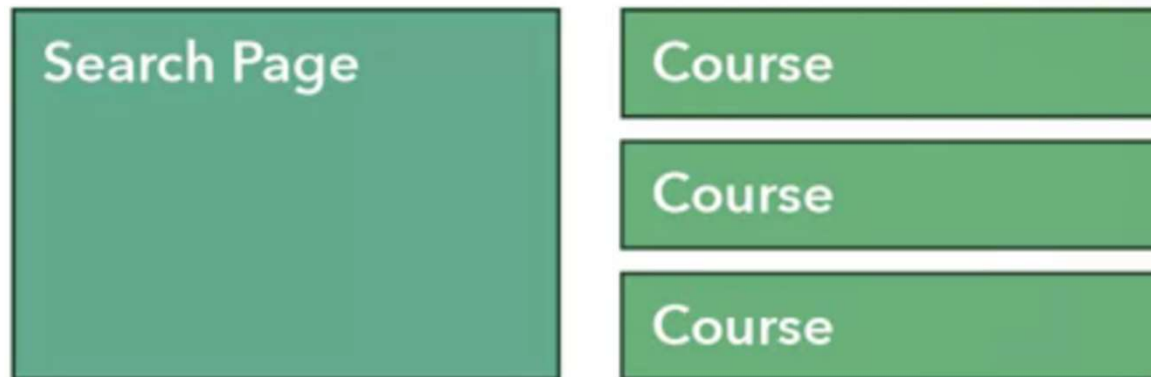


UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Conceptual Mockups

Conceptual mockups: Website Scenario

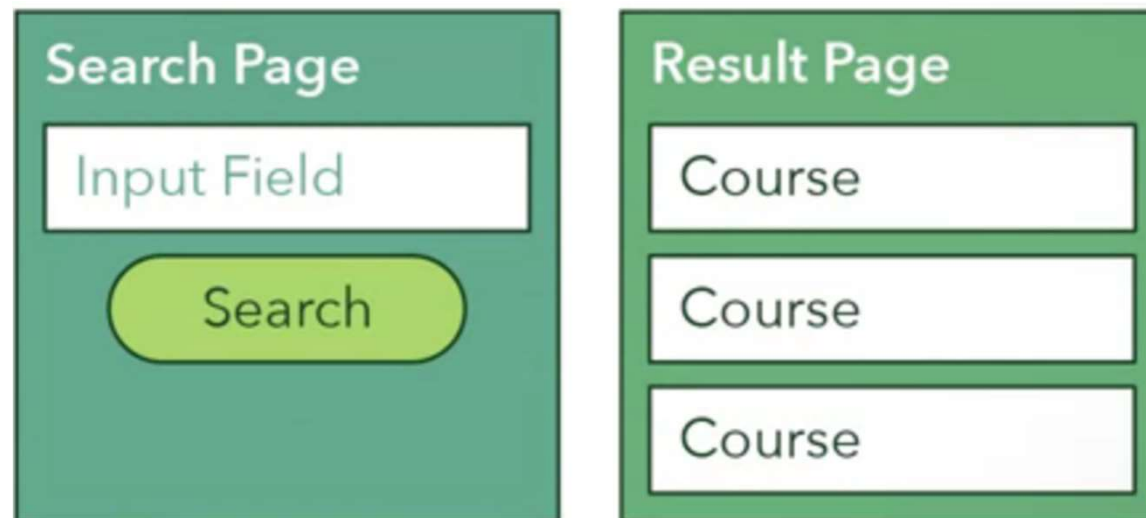
- Conceptual mockups provide your initial thoughts for how the requirements will be satisfied
- Example:
 - Design a task for a university course search website with the following requirement:
As a learner, I want to search for relevant courses through a search page
 - Hand drawn sketch or a drawing of the user interface



- The flaws in the initial mockup require further clarification with your client

Conceptual mockups: Website Scenario

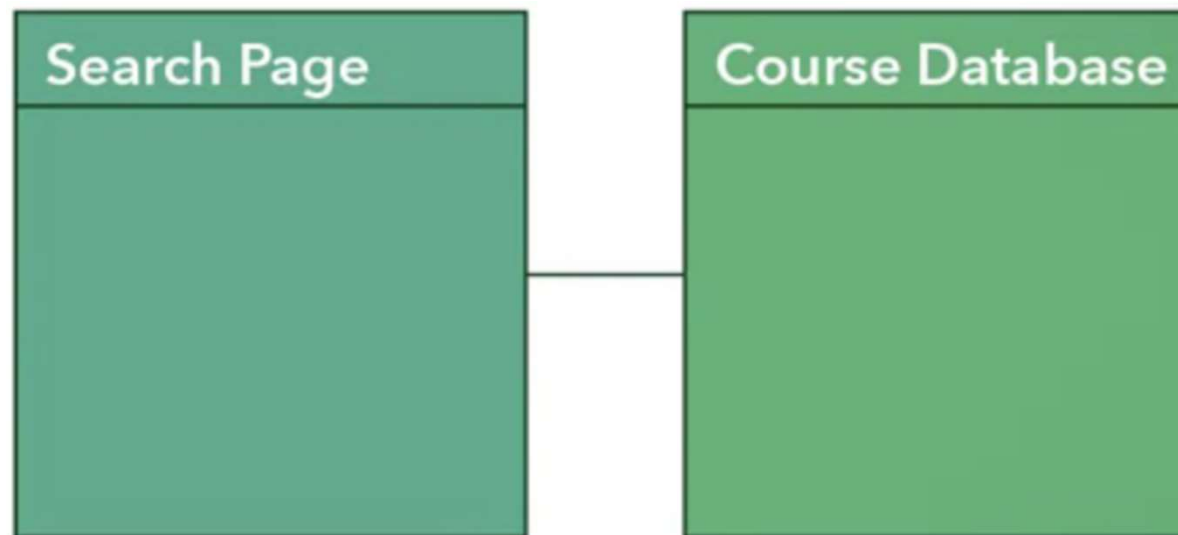
- Add missing components after the discussion with the client. For example, input field and search button



- From this mockup, we recognize many connections
- we refine each component until it is specific enough to be designed in detail

Conceptual mockups: Website Scenario

- Search Page requires Course Database in order to fulfill its responsibility of search, a connection exists between Search Page and Course Database



- Components turn into collections of functions, classes or other components. These pieces then represent a much simpler problem that the developers can individually implement



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Class, Responsibility and Collaborator (CRC) cards

Add technical details to the conceptual design

- How the components, connections and responsibilities formed in the conceptual design are further refined to give them **technical details**
- Class, Responsibility, Collaborator (CRC) is a technique for representing the technical information at a high level when forming the conceptual design
- CRC cards help you to organize your components into classes, identify the responsibilities and determine how they will collaborate with each other

CRC card

Class Name	
Responsibilities	Collaborators

- Collaborators are other classes that the class interacts with to fulfill its responsibilities
- CRC cards are cheap, editable and widely available and small on purpose

CRC card: Example

■ Bank machine example

- You insert your bank card into the bank machine. The bank machine will then ask you to enter a PIN authenticating you for access. After that, you can choose to deposit, withdraw or check your balances
- This scenario suggests the basic requirements for the system. It is an incomplete set of requirements but it's a good start

CRC card: Example

Bank Customer

Responsibilities

Collaborators



Bank Customer

Responsibilities

- Insert bank card
- Enter Pin code
- Choose operation

Collaborators

-Bank Machine

CRC card: Example

Bank Customer

Responsibilities

- Insert bank card
- Enter Pin code
- Choose operation

Collaborators

-Bank Machine

Bank Machine

Responsibilities

- Authenticate bank customer
- Display task options
- Deposit & withdraw?
- Check balances

Collaborators

-Bank Customer

CRC card: Example

Bank

--	--

Network

--	--

Bank Customer

--	--

Bank Machine

--	--



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Trade offs and compromises

Design decisions

- Trade-offs in different quality attributes such as performance, convenience and security
- The door lock example: Balances between security and both convenience and performance
- Competing concerns with software design
 - Usability vs. Security
 - Performance vs. Scalability
 - Quality vs. Time to market
- Establish what is good enough

Context and requirements

- Context is important to determine what choice of solution is right for the balance of qualities

- **Functional requirements** [Build the correct software]
 - Describe what the system or application is expected to do
 - E.g., a media app has a functional requirement of being able to download a full-length movie
 - A key quality to satisfy is simply correctness

Context and requirements

- **Non-functional requirements [Build the software correctly]**
 - Specify how well the system or application does what it does
 - Such requirements may describe how well the software runs in particular situations
 - E.g., the media app can have non-functional requirements to download a full-length movie at a specific speed and to play such a movie within a certain memory limit.
 - Beyond correctness, other qualities to satisfy include: Performance, resource usage, efficiency,
 - Another kind of non-functional requirement concerns how well the code of the software can evolve, such as: re-usability, flexibility, maintainability etc.

Functional vs. Non-functional requirements

- Both functional and non-functional requirements are important to satisfy and Nonfunctional requirements could become functional
- You'll need to discuss what is acceptable with the clients. The desired qualities will put constraints on your system's design.
- Strike a balance during design
 - Performance and maintainability: High performance code may be less clear and less modular making it less maintainable.
 - Security and performance: The extra overhead for high security may reduce performance
- Validation (functional) vs. Verification (non-functional)

Structure of a specification document, IEEE standard

1. Introduction
2. General description
3. Functional requirements
4. Specification of data structures
5. Specifications of external interfaces
6. Performance requirements
7. Development constraints



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Appendix-1: History

General information

- The meeting of the scientific committee of the North Atlantic Treaty Organization
 - Place and time: (Germany 1968)
 - Attendees: the world's leading programming specialists
 - Purpose: study the causes of what was beginning to be called the software crisis

- Symptoms of the crisis
 - Adequacy problem: computerized systems often do not meet the needs of users
 - Reliability problem: systems often break down
 - Modifiability problem: maintenance is often complex and costly.
 - Cost and delay: budgets exceeded; projects were delayed
 - Inappropriate user interface

General information

- It is necessary to make software production an engineering discipline, with a solid scientific basis. Hence the title:
software engineering: Set of methods, techniques and tools necessary to produce quality software

- Software engineering provides added value to:
 - Users (clients): by producing software that is adapted to their needs
 - Managers: by reducing the cost of production and maintenance
 - Project managers: by making the software productive within a reasonable time

General information

- Production of quality software
 - Improve the development process
 - Improve languages
 - Use development tools (Programming environments, Software engineering workshops, ...)
 - Break down the development into several stages



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Appendix-2: Development life cycle stages

1- Preliminary study

■ Goals:

- Clarify the purpose of the system
- Dialogue with the different types of users
- Sketch out solutions
- Estimate the costs and deadlines
- Prepare a file for decision

■ Results:

- 1st version of the development contract, 1st estimate of the development cost
- Specifications, rough operating mode of the system
- General project plan

■ Means:

- Interview plan with the customer (interview techniques)
- Review to validate and approve the exhibited proposals

1- Preliminary study

- Specifications (Cahier des charges)
 - Result of the preliminary studies carried out, directly or indirectly by the project owner
 - Give all the necessary elements to allow the project manager to elaborate and cost solutions
 - Remain strictly at the level of the expression of the need and the functional requirements, without imposing the choice of design
- Organization of the specifications
 - Technical clauses
 - > Functionalities of the software to be realized
 - > Constraints to be taken into account
 - Quality clauses
 - General clauses (legal, financial,)

2- Requirements phase (Specifications)

■ Goals:

- Define the "what" -> Obtain a precise and unambiguous description of the software system
- For the Software Product: define the required performances, the software functions, the real world objects manipulated, the software interfaces with its environment, the inputs and outputs
- For the Project: define the resources, the constraints of realization

■ Results:

- User manual, Specification document
- Detailed plan of the rest of the project, Refinement of the cost estimate

3- General Design

- **Goals: Describe the architecture of the software**
 - Obtain a decomposition of the system into a set of modules
 - Obtain a precise description of the role of each module.
 - Verify feasibility and traceability to the specification: trace (link) between the system functions and the software modules.

- **Results**
 - Design document: description of the modular structure of the system
 - Definition of modules and interfaces.
 - Update of the user manual

- **Means**
 - Methods, languages, design tools: UML, structured design, ...
 - Design reviews

4- Detailed Design

■ Goals

- To obtain a detailed description of the Modules, processes and data structures allowing immediate coding

■ Results

- Detailed design file
- Unit test planning

■ Means

- Pseudo-codes
- Programming languages (Abstract)

5- Development

■ Goals

- Obtain programs: translate the detailed design into a programming language following the defined standards.

■ Results

- Programs
- Technical documentation

■ Means

- Programming languages. Syntax editors, ...
- Code reviews

6- Unit Testing

■ Goals

- Test each module separately: test the module body and compare its behavior to a baseline behavior

■ Results

- Test results
- Errors report

■ Means

- Static analyzers, dynamic tests, test tools

7- Integration and verification

■ Goals

- Recompose the software into modules from its components
- Verify the functionality and performance of the complete system
- Verification of the respect of the programming standards
- Verification of the documentation

■ Results

- Integrated software system
- Internal and external documentation

■ Means

- Use of test sets
- Performance evaluation tools

8- Validation

■ Goals

- To test the software under conditions representative of its operational use
- Validate the software against the requirements specification

■ Results

- Validation documents. Qualification report, quality-reliability report
- Certified and installed program

■ Means

- Use of test sets
- Quality control

9- Operation & Maintenance

- This phase is **not** part of the project itself, but it always turns out that modifications must be made to the product:
 - Correction of errors detected during operation
 - Modification of the software to meet new needs
 - Optimization of certain functions

- Waterfall vs. Agile?



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Appendix-3: Technical Design - House Scenario

Conceptual Design: House Scenario

■ Conceptual mockups

- Initial thoughts for how the requirements will be satisfied
- Define **components**: the land, the house itself, the kitchen, the gym, the bathroom, the bedrooms and the living room
- Define **connections**: For example, if the living room is openly accessible from the kitchen, the living room has a connection to the kitchen
- Each component has a task it needs to perform, known as **responsibility**. For example, the kitchen's main responsibility is to provide space for storing kitchenware, food supplies, and water for meal preparation

Note: how we don't mention specifics about wiring and plumbing

Technical Design: House Scenario

- Technical details of each component
 - Splitting components into smaller and smaller components that are specific enough to be designed in detail. For example, the gym component will require further components like a floor. The floor will be responsible for supporting a lot of weight
 - Compromises might arise when creating an acceptable solution. What if reinforcing the floor of the gym requires putting in columns or beams in the basement below the gym?
 - Rework the conceptual design if it is proved impossible to achieve in the technical design or fail to meet the requirements



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Appendix-4: Requirements

Non-functional requirements

■ Performance requirements

- Required response time (to a request for example)
- Size of the manipulated information and memory space required
- Restrictions on data representation
- Average number of transactions per second
- Number of terminals connected to the system

■ Design constraints

- Use of standard methods
- Programming language
- Operating system
- Hardware
- Documentation structure (report format)
- Example design constraint: “The system development process and documentation shall comply with DEF-STAN OO-65.”

Functional vs. Non-functional requirements

- Consider an analogy involving categories of cars:
 - Both sports cars and minivans meet the functional requirement of providing transportation
 - But each category, indeed each model of car, offers a different balance of achieved non-functional requirements with different factors like acceleration, handling, cargo capacity, weight and fuel economy
- The required quality should be verified through techniques like reviews and tests. As well, certain qualities can be validated with feedback from end users

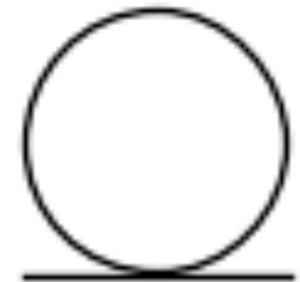


UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Appendix-5: Categories of Objects in Design

Entity objects

- Real-world entity in the problem space
- Know attributes about themselves
- Modify themselves, and have some rules for how to do so
- Initial choice when identifying objects
- Usually represent data storage
- Model in Model–View–Controller (MVC) software design pattern



Boundary objects

- Sit at the boundary between systems
- Deal with other another external software systems, the internet or a user
- Show information to the user and getting their input (user interface)
- View in Model–View–Controller (MVC) software design pattern



Control objects

- Responsible for coordination
- Represent the transfer of information
- Mediator: it simply coordinates the activities of many different objects so that they can stay loosely coupled
- Controller in Model–View–Controller (MVC) software design pattern



References and Acknowledgments

- UML 2 and the Unified Process: Practical Object-Oriented Analysis and Design
- Design Patterns: Elements of Reusable Object-Oriented Software
- Timeless Way of Building, Alexander, ISBN 0-19-502402-8
- Software Design and Architecture by Kenny Wong, University of Alberta Computing Science, Faculty of Science, Coursera
- A Pattern Language, Alexander, 0-19-501-919-9
- Design Patterns, Gamma, et al., 0-201-63361-2 CD version 0-201-63498-8
- Pattern-Oriented Software Architecture, Vol. 5, Buschmann, et al., 0-471-48648-5
- Concurrent Programming in Java, 2nd ed., Lea, 0-201-31009-0
- Applying UML & Patterns, 2nd ed., Larman, 0-13-092569-1
- “Object-Oriented Patterns,” 2004, P. Coad; Comm. of the ACM, 9/92
- “Documenting Frameworks using Patterns,” R. Johnson; OOPSLA '92
- A Lightweight Guide to the Theory and Practice of Scrum, Version 2.0 . Pete Deemer, Gabrielle Benefield, Craig Larman, Has Vodde
- <https://tms-outsource.com/blog/posts/scrum/>
- Scrum Handbook, Jeff Sutherland, 2019

References and Acknowledgments

- The Art of Agile Development 1st Edition, by James Shore and Shane Warden
- Essential Scrum: A Practical Guide to the Most Popular Agile Process, Kenneth S. Rubin's Essential Scrum, 2012
- Scrum: The Art of Doing Twice the Work in Half the Time, Jeff Sutherland, 2014
- The Scrum Field Guide: Practical Advice For Your First year, Mitch Lacey, 2012
- Agile Retrospectives: Making Good Teams Great by Esther Derby & Diana Larsen
- User Story Mapping: Discover the Whole Story, Build the Right Product, 2021
- The Deadline: A Novel about Project Management
- Agile Leadership Toolkit by Peter Koning, 2019
- Akhtar, A., Bakhtawar, B. and Akhtar, S., 2022. EXTREME PROGRAMMING VS SCRUM: A COMPARISON OF AGILE MODELS. International Journal of Technology, Innovation and Management (IJTIM), 2(2).
- Ashari, I.F., Aryani, A.J. and Ardhi, A.M., 2022. Design and Build Inventory Management Information System Using the Scrum Method. JSil (Jurnal Sistem Informasi), 9(1), pp.27-35.



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

End of Lecture